Lecture 10

Generic

Definition

- **Generics** is a programming approach that allows an object to operate with multiple different data types.
- The term **generics** means *type parameterization*.
  - It allows us to create and use a class, interface, or method with various data types.
- A class, interface, or method that *operates on a specified parameterized type* is called **generic**.

Non-generic approach

- **Example:** Using `ArrayList` with different data types:

```
ArrayList myList = new ArrayList();

myList.add(10);
myList.add("Hello");
myList.add(true);
myList.add(15.75);
```

When retrieving data:

```
int a = (Integer) myList.get(0);
String str = (String) myList.get(1);
```

Generic approach

- **Example:** Using `ArrayList` with `Integer` data type

```
ArrayList<Integer> myList =
        new ArrayList<Integer>();

myList.add(10);
myList.add("Hello"); // error
myList.add(true); // error
myList.add(15);
```

Generic

When retrieving data:

```
int a = myList.get(0);
```

Generic approach

- **Example:** Using `ArrayList` with `String` data type

```
ArrayList<String> myList =
        new ArrayList<String>();

myList.add("Hello");
myList.add("Goodbye");
```

Generic

When retrieving data:

```
String hello = myList.get(0);
```

Diamond Operator <>

- Since Java 7, the type argument on the right side can be omitted using the <> (diamond) operator — the compiler infers the type automatically.

```
// Before Java 7 — must repeat type argument on both sides
Pair<Integer, String> p1 = new Pair<Integer, String>(1, "One");

// Java 7+ — compiler infers <Integer, String> from left side
Pair<Integer, String> p2 = new Pair<>(2, "Two");
```

Advantages of generics

- Type checking at compile time
  - The Java compiler performs type checking on generic code to detect issues such as type safety violations.
  - Fixing errors at compile time is much easier than fixing errors at runtime.

Advantages of generics

- No need for type casting
  - The following code snippet does not use generics and requires type casting:

```
List list = new ArrayList();
list.add("hello");
String s = (String) list.get(0); // type casting required
```

- When using generics, type casting is unnecessary:

```
List<String> list = new ArrayList<>();
list.add("hello");
String s = list.get(0); // no type casting required
```

Advantages of generics

- Allows programmers to implement generic algorithms.
  - By using generics, programmers can implement generic algorithms with various optional data types.
  - The code becomes clearer and easier to understand.

Creating generic classes and methods

- Generics type parameter naming convention

Character	Meaning
E	Element
K	Key
V	Value
T	Type
N	Number
S, U	2nd, 3rd type (e.g. `Pair<T, S>`)
R	Result (e.g. `Function<T, R>`)

Creating generic classes

- Creating a generic class with one type parameter:

```
public class GenericType<T> {
    private T t;

    public T get() {
        return this.t;
    }

    public void set(T t1) {
        this.t = t1;
    }
}
```

Using generic classes

- Using a generic class with one type parameter:

```
GenericType<String> obj1 = new GenericType<>();
obj1.set("Pancake"); // valid
obj1.set(10); // invalid

GenericType obj2 = new GenericType(); // no generic
obj2.set("Pancake"); // valid

obj2.set(10); // still valid
System.out.println(obj2);
```

Raw types and unchecked warnings

- A **raw type** is a generic class/interface used without a type argument.
- The compiler issues an **unchecked warning** because type safety cannot be guaranteed.

```
// Raw type → compiler warning: unchecked call
GenericType rawObj = new GenericType();
rawObj.set("Hello");  // ⚠ unchecked warning
rawObj.set(42);   // ⚠ also compiles but not type-safe!
// Fix: always specify the type argument
GenericType<String> typedObj = new GenericType<>();
```

Example: Creating a generic class

- Creating a generic class with **two** type parameters:

```
public class Pair<T, S> {
    private T first;
    private S second;

    public Pair(T fi, S se) {
        first = fi;
        second = se;
    }

    public T getFirst() { return first; }

    public S getSecond() { return second; }
}
```

Example: Using a generic class

- Using a generic class with **two** type parameters:

```
public static void main(String[] args) {
    Pair<Integer, String> p1 = new Pair<>(1, "One");
    Pair<String, String> p2 = new Pair<>("Hello", "World");
    System.out.println(p1.getFirst() + ", " + p1.getSecond());
    System.out.println(p2.getFirst() + ", " + p2.getSecond());
}
```

Limiting data types

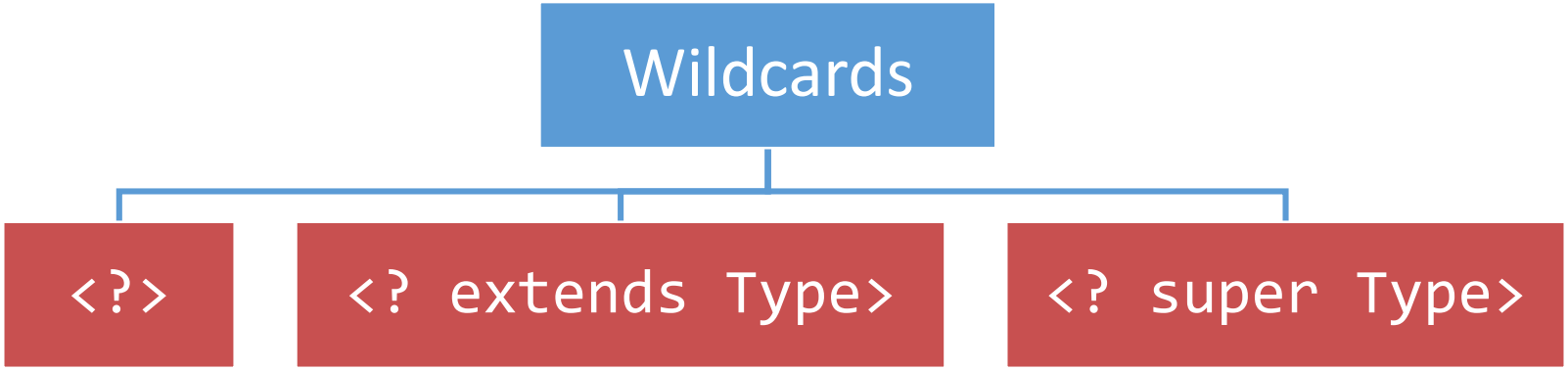
```
public class GenericB<T extends Number> {
    private T num;

    GenericB(T n) {
        num = n;
    }

    double reciprocal() {
        return 1 / num.doubleValue();
    }

    double fraction() {
        return num.doubleValue() % num.intValue();
    }
}
```

Limiting `T` to subtypes of `Number`

Example: Limiting data types

```
GenericB<Integer> e1 = new GenericB<Integer>(5);
System.out.println("Reciprocal: " + e1.reciprocal());

System.out.println("Fraction: " + e1.fraction());

GenericB<Double> e2 = new GenericB<Double>(7.5);
System.out.println("Reciprocal: " + e2.reciprocal());

System.out.println("Fraction: " + e2.fraction());

GenericB<Float> e3 = new GenericB<Float>(7.25f);
System.out.println("Reciprocal: " + e3.reciprocal());

System.out.println("Fraction: " + e3.fraction());
```

Generic placeholder symbols



Example: not using placeholder symbols

```
public class MyNumber<T extends Number> {
    private T val;

    MyNumber(T v) {
        val = v;
    }

    double reciprocal() { return 1 / val.doubleValue(); }

    double fraction() {
        return val.doubleValue() % val.intValue();
    }

    boolean testEquals(MyNumber<T> e) {
        return val.doubleValue() == e.num.doubleValue();
    }
}
```

Problem when not using placeholder symbols

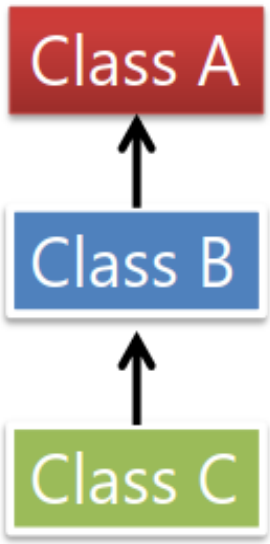
```
MyNumber<Integer> n1 = new MyNumber<>(5);
MyNumber<Double> n2 = new MyNumber<>(5.0);
// the following line has error because
// n1.testEquals only receives MyNumber<Integer>
System.out.println("n1==n2 ? " + n1.testEquals(n2));
```

Symbol: <?>

```java
public class MyNumber<T extends Number> {
    private T val;
    MyNumber(T v) { val = v; }
    double reciprocal() { return 1 / val.doubleValue(); }
    double fraction() {
        return val.doubleValue() % val.intValue();
    }

    boolean testEquals(MyNumber<?> e) {
        return val.doubleValue() == e.val.doubleValue();
    }

    public static void main(String[] args) {
        MyNumber<Integer> n1 = new MyNumber<>(5);
        MyNumber<Double> n2 = new MyNumber<>(5.0);
        // now it's working
        System.out.println("n1==n2 ? " + n1.testEquals(n2));
    }
}
```

Symbol: `<? extends Type>`

```
public void processEle(List<? extends A> elements){
    ...
}
```




```
List<A> listA = new ArrayList<A>();
processEle(listA);

List<B> listB = new ArrayList<B>();
processEle(listB);

List<C> listC = new ArrayList<C>();
processEle(listC);
```

Symbol: `<? super Type>`

```
public static void processEle(List<? super A> elements) {
    for (Object obj : elements) {
        System.out.println(obj);
    }
}

public static void main(String[] args) {
    List<A> listA = new ArrayList<>();
    processEle(listA);

    List<Object> listO = new ArrayList<>();
    processEle(listO);
}
```

Generic constructor

```java
public class Summation {
    private int sum;

    <T> Summation(T size) { // generic constructor
        sum = 0;
        int n = Integer.parseInt(size.toString());
        for (int i = 0; i < n; i++) {
            sum += i;
        }
    }

    int getSum() {
        return sum;
    }

    public static void main(String[] args) {
        Summation obj = new Summation(4);
        System.out.println("Sum of 1->4 is: " + obj.getSum());
    }
}
```

- **Generic constructor:** a constructor that is defined with its own type parameters.
- Independent of whether its class is generic or not.

Generic constructor with placeholder symbol

```
public class Summation {
    private int sum;

    <T extends Number> Summation(T size) {
        sum = 0;
        for (int i = 0; i < size.intValue(); i++) {
            sum += i;
        }
    }

    int getSum() {
        return sum;
    }

    public static void main(String[] args) {
        Summation obj = new Summation(4.0);
        System.out.println("Sum of 1->4 is: " + obj.getSum());
    }
}
```

Generic method

- Following is an example of the **definition** and **usage** of a generic method:

```
public static <T extends Comparable<T>> T max(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}
// One method works for Integer, Double, String...
System.out.println(max(3, 7));  // 7
System.out.println(max("apple", "mango")); // mango
```

Multiple type bounds

```
public class NumberBox<T extends Number & Comparable<T>> {
    private T num;

    NumberBoxT n) {
        num = n;
    }

    double reciprocal() {
        return 1 / num.doubleValue();
    }

    double isGreaterThan(T other) {
        return num.compareTo(other) > 0;
    }
}
```

T must extend `Number` **AND** implement `Comparable<T>`

Generic constructor example

…with multiple type bounds

```
<T extends Number & Serializable> Summation(T size) {
    sum = 0;
    for (int i = 0; i <= size.intValue(); i++) {
        sum += i;
    }
}
```

Generic method example

…with multiple type bounds

```
public <E extends Number & Comparable<E>>
            void method1(E param) {

}
```

Generic interfaces

```
interface Containment<T> {
    boolean contains(T obj);
}

public class MyClass<T> implements Containment<T> {
    T[] arrayRef;
    public MyClass(T[] arr) {
        arrayRef = arr;
    }
    public boolean contains(T obj) {
        for (T x : arrayRef)
            if (x.equals(obj)) return true;
        return false;
    }
}
```

Generic interfaces

```
public static void main(String[] args) {
    Integer x[] = {1, 2, 3};
    MyClass<Integer> ob = new MyClass<Integer>(x);
    Integer a = 20;
    if (ob.contains(a)) {
        System.out.println(a + " in array");
    } else {
        System.out.println(a + " NOT in array");
    }
}
```

Type Erasure

- **Type Erasure** is how Java implements generics at the bytecode level.
  - At compile time, the compiler checks and enforces generic type rules.
  - At runtime, all type parameters are **erased** — replaced by `Object` (or the upper bound if specified). No generic type info exists in the `.class` file.
- Therefore, at runtime:
  - `List<String>` and `List<Integer>` are both just `List`
  - `new ArrayList<String>().getClass() == new ArrayList<Integer>().getClass()` → **`true`**

Type Erasure — consequences

- Because of Type Erasure, several operations on generic types are restricted at runtime:
  - **Cannot use `instanceof` with a parameterized type:**

    `obj instanceof List<String>` is a compile error; use `obj instanceof List<?>` instead
  - **Cannot create arrays of generic types:**

    `new List<String>[10]` is illegal (use `List<?>[]` instead)
  - **Cannot call `new T()`** — the JVM has no information about `T` at runtime

    → Pass `Class<T>` as a parameter instead
  - **Cannot use static fields of type `T`** — all instances share the same static field, so `T` would be ambiguous

Limitations of generics

- Unable to initialize generics with primitive data types

```
// incorrect
Pair<int, char> p1 = new Pair<>(8, 'a');

// correct
Pair<Integer, Character> p2 = new Pair<>(8, 'a');
```

- Cannot create an instance for data type

```
public class Gen<T> {
    T obj;
    Gen() {
        obj = new T(); // illegal
    }
}
```

Limitations of generics

- Cannot be static within a class
  - The meaning of `T` in different instances of `Gen` may be different.
  - However, static variables / methods do not get separate copies for different instances.

    (In other words, there's only **1** shared copy of a static member per class)

```
public class Gen<T> {
    static T obj; // error

    static T getObj() { // error
        return obj;
    }
}
```

Limitations of generics

- Cannot create arrays

```
// error: Generic array creation
Gen<Integer> arr1[] = new Gen<Integer>[10];

Gen<?> arr2[] = new Gen<?>[10]; // ok
arr2[0] = new Gen<Integer>(24);
arr2[1] = new Gen<String>("Hello");
```

- Limitations on generic exceptions
  - A generic class **cannot inherit** from the `Throwable` class.
  - Therefore, we cannot create a generic exception class.

Summary

- Definition of generics
- Advantages of generics
- Creating generic classes & methods
- Diamond operator `<>`, raw types & unchecked warnings
- Bounded type parameters & multiple bounds
- Wildcards: `<?>`, `<? extends T>`, `<? super T>`
- Generic constructors, methods & interfaces
- Type Erasure
- Limitations of generics